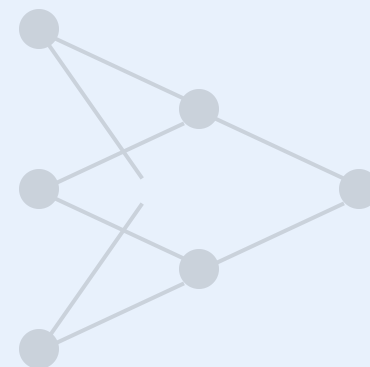


출력의 책임을 입력으로 되돌려 보내는 수학의 원리

거꾸로 배우는 지혜: 역전파와 연쇄 법칙



강의 목표

- ✓ 출력층의 오차가 각 가중치에 어떻게 배분되는지 이해합니다.
- ✓ 복잡한 미분을 단순화하는 '연쇄 법칙'과 계산 그래프의 직관을 습득합니다.
- ✓ 역전파를 통해 딥러닝 학습이 빠르고 효율적으로 이루어지는 원리를 파악합니다.

오차를 알았다면, 이제 "누가, 얼마나 잘못했는가?"

02

⚠ 문제 상황

최종 손실(Loss)이 발생했습니다.

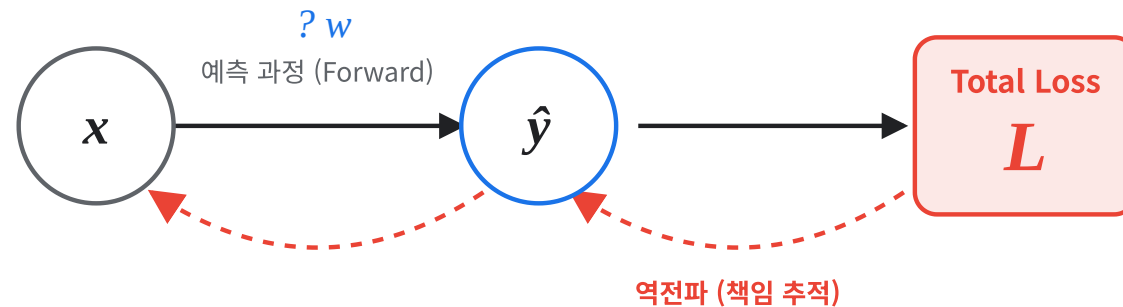
하지만 입력에 연결된 여러 가중치(w_1, w_2, w_3) 중 누구의 책임이 가장 큰지 알 수 없습니다.

⚖️ 책임의 크기 = 기울기

역전파는 이 책임을 수학적으로 계산합니다.

기울기가 클수록(화살표가 굵을수록) 오차에 대한 책임이 크다는 뜻이며, 가중치를 더 많이 수정해야 합니다.

"너 때문이야!" (Gradient)



≡ 책임 분석표 (Gradient Report)

가중치	기울기 (책임도)	책임의 크기	조치
w_1	-5.2		매우 큼 (대폭 수정)
w_2	-0.1		미미함 (유지)
w_3	-2.3		중간 (적당히 수정)

학습의 핵심: 가중치 업데이트

03

🎯 학습 목표

손실 함수 L 을 **최소화**하기 위해 파라미터 w 를 어느 방향으로, 얼마나 움직일 것인가?

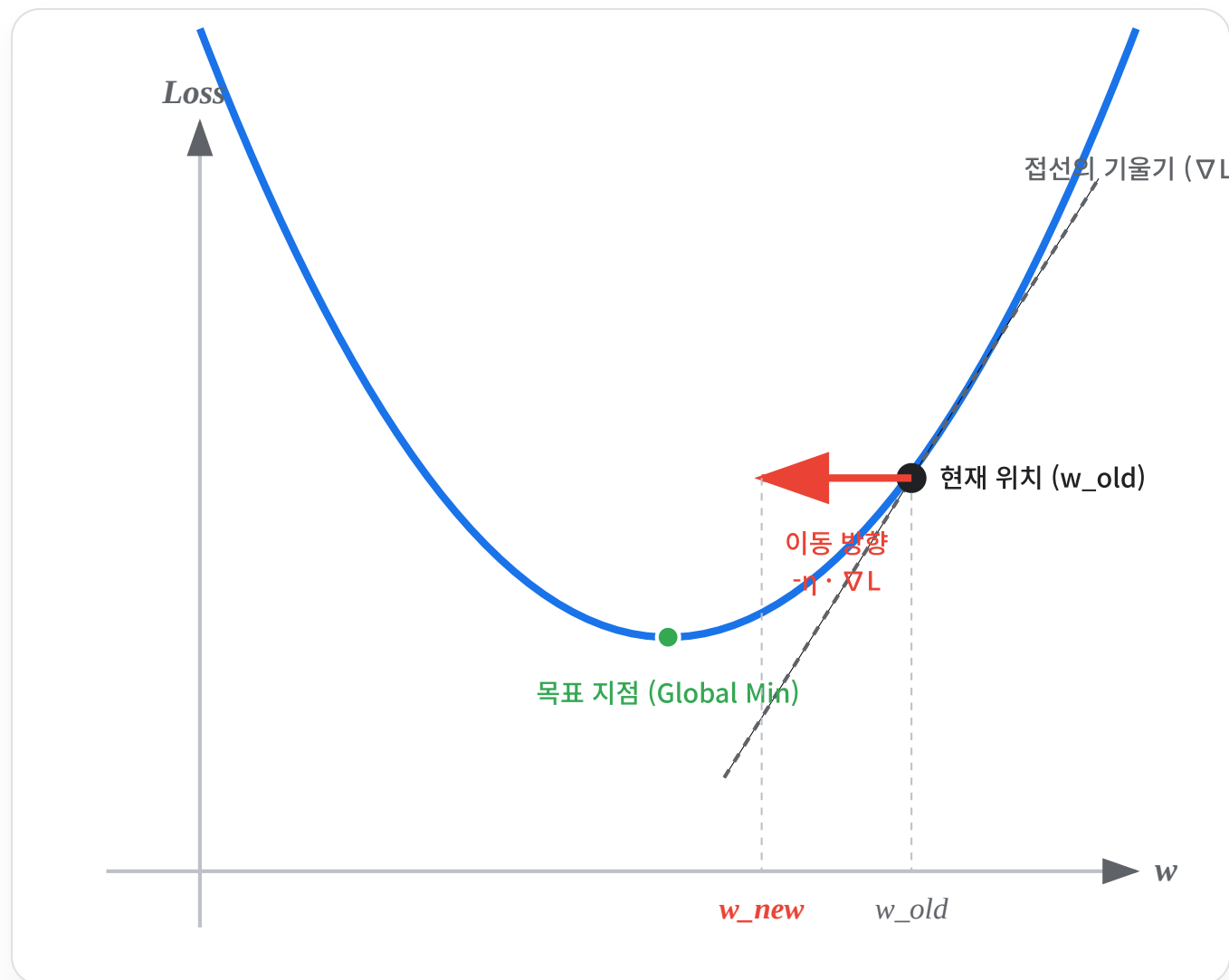
$$w^{\text{new}} = w^{\text{old}} - \eta \cdot \frac{\partial L}{\partial w}$$

(새로운 위치 = 현재 위치 - 학습률 $\times$ 기울기)

🔍 직관적 해석

- $\partial L / \partial w > 0$ (오르막): w 를 **감소**시켜야 함
- $\partial L / \partial w < 0$ (내리막): w 를 **증가**시켜야 함

👉 기울기의 **반대 방향**으로 이동하여 골짜기(최소점: Global Min)를 찾습니다.



미분(Derivative)의 직관: '민감도'

04

🔍 미분의 본질적 질문

"특정 가중치 w 를 아주 조금 변화시켰을 때(Δw),
전체 오차 L 은 얼마나 변하는가(ΔL)?"

$$\frac{\partial L}{\partial w} \approx \frac{\Delta L}{\Delta w}$$

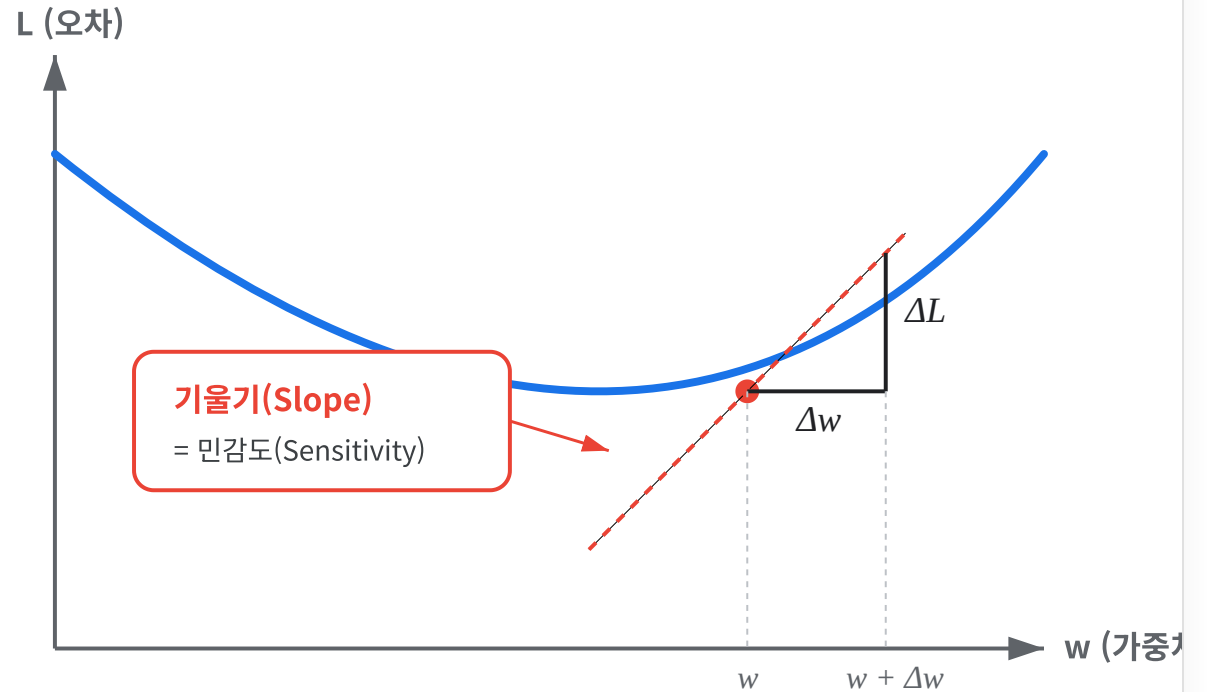
📊 민감도(Sensitivity) 해석

미분값이 크다: 이 가중치는 오차에 큰 영향을 미침.

→ '책임'이 크므로 많이 수정해야 함.

미분값이 0에 가깝다: 이 가중치는 오차와 무관함.

→ 수정할 필요가 거의 없음.



계산 그래프(Computational Graph) 도입

05

계산 그래프란?

복잡한 수식을 **노드(Node)**와 **엣지(Edge)**로 단순화하여 표현한 지도입니다.

노드(○): 연산 (덧셈, 곱셈 등)

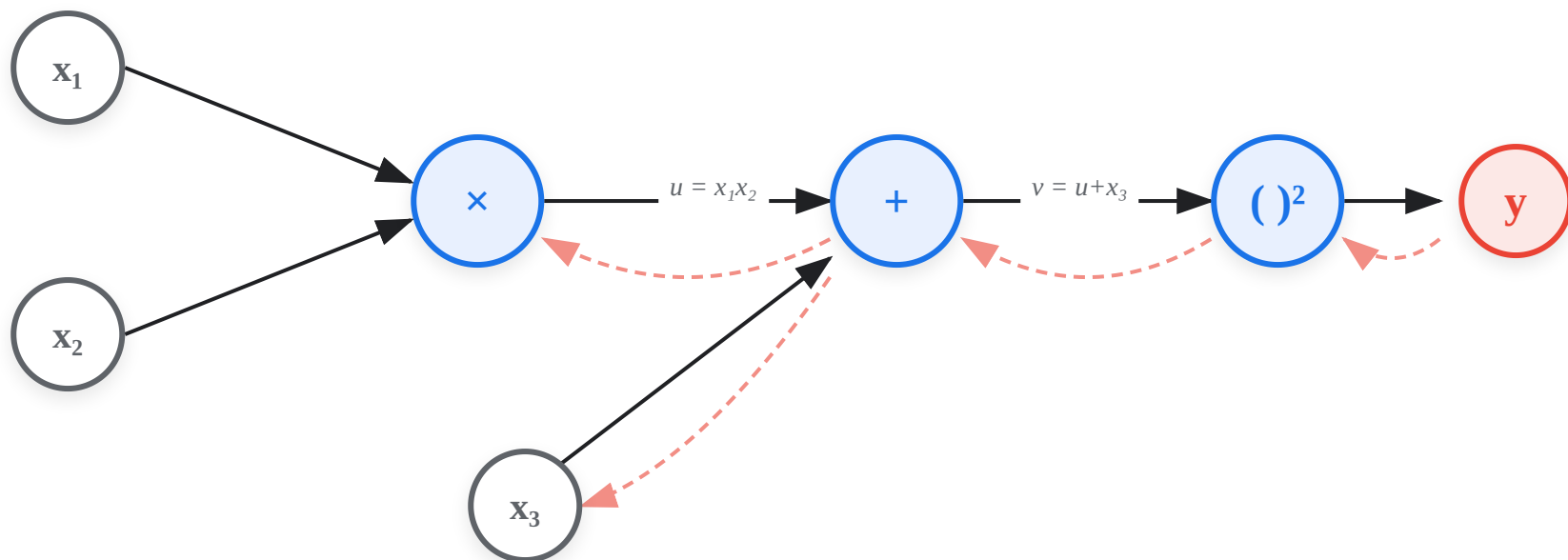
엣지(—): 흐르는 데이터 (값)

왜 사용하는가?

전체를 한 번에 미분하는 대신, 각 노드 별로 '국소적 미분'을 계산하고 이를 연결하면 되기 때문입니다.

EXAMPLE FORMULA

$$y = (x_1 \cdot x_2 + x_3)^2$$



— 순전파 (Forward) - - - 역전파 (Backward)

연쇄 법칙(Chain Rule)의 원리

수학적 정의

$$\frac{dL}{dx} = \frac{dL}{dy} \cdot \frac{dy}{dx}$$

합성 함수의 미분은 각 단계의 국소적 미분(Local Derivative)들을 곱한 것과 같습니다.

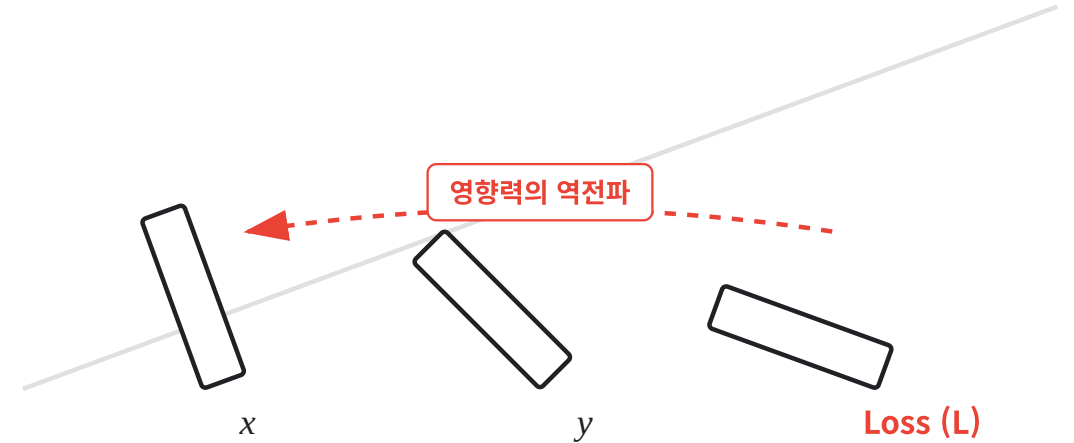
"전체 변화율 = 부분 변화율들의 곱"

핵심 아이디어

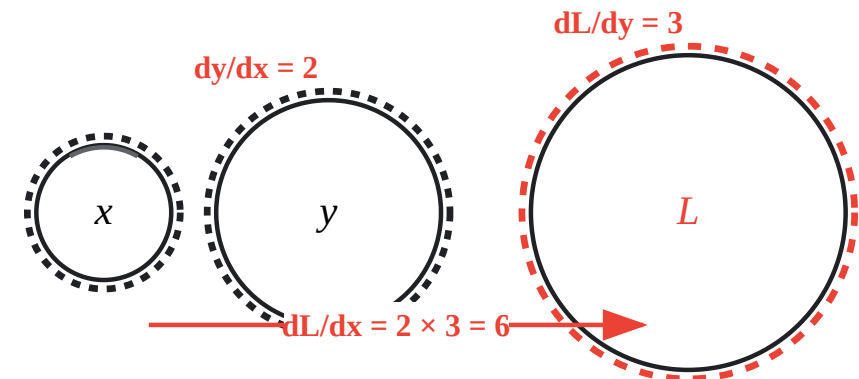
한 번에 계산하기 어려운 거대한 미분도, 잘게 쪼개서 곱하면 쉽게 구할 수 있습니다.



도미노
효과



기어
장치



국소적 미분(Local Gradient): 노드별 규칙

국소적 미분이란?

전체 네트워크가 아무리 복잡해도, 각 노드는 **자신에게 들어온 입력만 보고** 미분값을 계산할 수 있습니다.

이 값은 순전파(Forward Pass) 때 미리 계산해 둘 수 있으며, 역전파가 오면 단순히 **곱해서** 뒤로 넘겨줍니다.

계산 전략

- 1. **순전파**: 입력값(x, y)을 저장
- 2. **국소 미분**: 공식에 대입하여 계산
- 3. **역전파**: 상류층 미분($\partial L / \partial z$) $\times$ 국소 미분

노드 타입	순전파 (수식)	국소 미분 (결과)
+ 덧셈 분배기 $z = x + y$ →	1, 1	
× 곱셈 스위치 $z = x \cdot y$ →	y, x	
x^2 제곱 $z = x^2$ →	2x	
σ Sigmoid $a = \sigma(z)$ →	a(1-a)	
ReLU ReLU $a = \max(0,z)$ →	1 (z>0) 0 (z≤0)	

예제: $y = (x_1 \cdot x_2 + x_3)^2$

역전파 흐름: 숫자가 거꾸로 타고 넘는 단계별 시연

08

Step 0: 출력층

오차 출발

$$\partial L / \partial y = 1$$

손실 함수(L) 자신에 대한 미분은 항상 1입니다. 여기서 역전파가 시작됩니다.

Step 1: 제공 노드

국소 미분: 2s

$$\partial L / \partial s = \partial L / \partial y \cdot 2s = 1 \cdot (2 \times 7) = 14$$

s=7일 때, 미분값 14가 뒤로 전달됩니다.

Step 2: 덧셈 노드

국소 미분: 1 (그대로 전달)

$$\partial L / \partial m = 14 \cdot 1 = 14$$

$$\partial L / \partial x_3 = 14 \cdot 1 = 14$$

덧셈 노드는 상류에서 온 미분(14)을 양쪽으로 그대로 복제하여 전달합니다.

Step 3: 곱셈 노드

국소 미분: 상대방 값

$$\partial L / \partial x_1 = 14 \cdot x_2 = 14 \cdot 3 = 42$$

$$\partial L / \partial x_2 = 14 \cdot x_1 = 14 \cdot 2 = 28$$

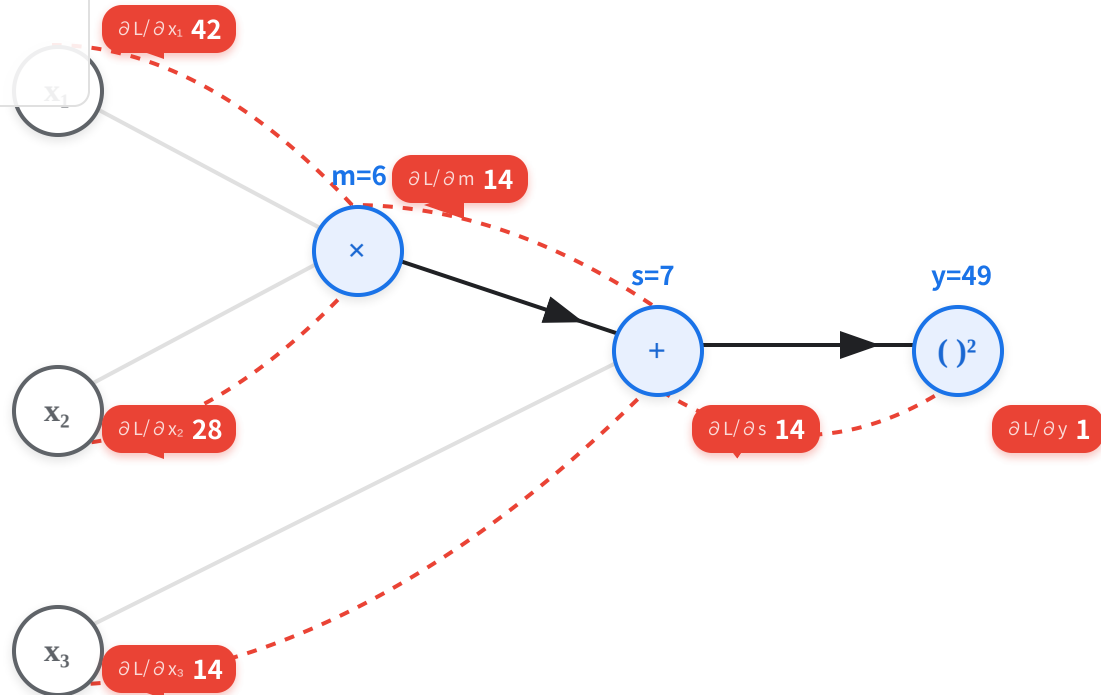
곱셈 노드는 들어온 미분값에 '서로 바꾼 입력값'을 곱해서 전달합니다.

INPUT VALUES

$$x_1 = 2$$

$$x_2 = 3$$

$$x_3 = 1$$

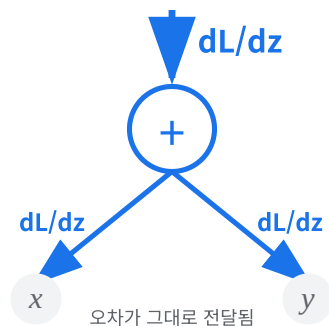


← 빨간 화살표를 따라 미분값이 역류합니다.

덧셈 vs 곱셈 노드의 역전파

+

덧셈 노드 (Addition)



$$z = x + y$$

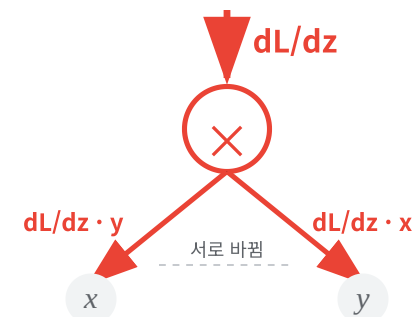
$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot 1$$

📡 그대로 전달 (Distributor)

덧셈 노드는 상류에서 흘러온 미분값을 **변형 없이 그대로 복제**하여 모든 하류 노드에 전달합니다. 입력값의 크기에 영향을 받지 않습니다.

×

곱셈 노드 (Multiplication)



$$z = x \cdot y$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot y$$

🔗 값 스위칭 (Switcher)

곱셈 노드는 상류의 미분값에 **순전파** 때의 '상대방 입력값'을 곱해서 전달합니다. 큰 입력값 쪽으로 더 큰 기울기가 전달됩니다.

활성화 함수 노드의 역전파

역전파 수식

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \cdot \phi'(z)$$

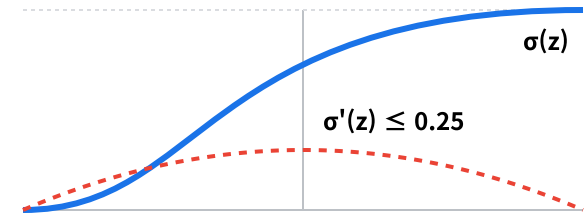
활성화 함수 노드를 지날 때, **미분값** $\phi'(z)$ 가 곱해 집니다. 이 값이 1보다 작으면 오차가 줄어들고, 0에 가까우면 신호가 사라집니다.

⚠ 기울기 소실 주의:

계속해서 1보다 작은 값이 곱해지면, 입력층에 도달할 때 오차 신호는 0이 됩니다.

Sigmoid

출력: (0, 1)
최대 미분값: 0.25



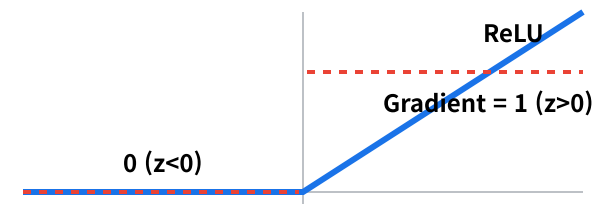
Tanh

출력: (-1, 1)
최대 미분값: 1.0



ReLU

출력: $[0, \infty)$
미분값: 0 또는 1



기울기 소실(Gradient Vanishing) 다시 보기

⚠ 원인: 연쇄 법칙의 곱셈 지옥

역전파는 국소 미분값들을 계속 곱하면서 입력층으로 이동합니다.
만약 이 값들이 1보다 작다면? (예: 0.25)

$$\text{Total Gradient} = 0.25 \times 0.25 \times \dots \times 0.25 \approx 0$$

☹ 결과: 학습 정체

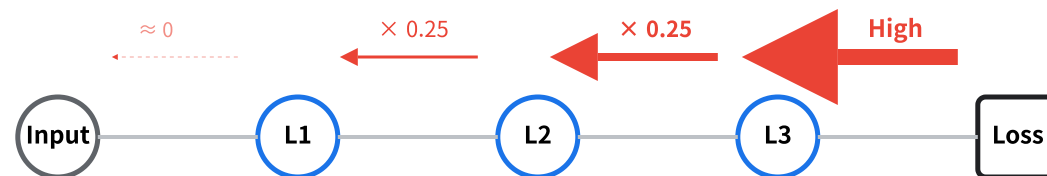
입력층에 가까운 가중치들은 업데이트 신호(기울기)를 거의 받지 못해 학습되지 않습니다. 깊은 신경망(Deep Network)을 쌓아도 성능이 좋아지지 않는 주원인입니다.

✅ 해결 전략: "곱해도 작아지지 않게"

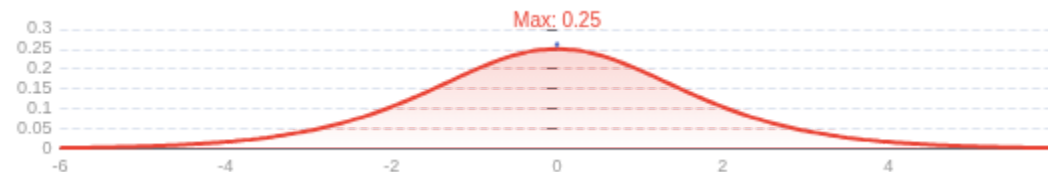
ReLU: 미분값이 양수 영역에서 항상 1 (사라지지 않음)

ResNet: 뎛셈 연결(Skip Connection)로 신호 보존

시그모이드 층을 통과하며 사라지는 기울기



왜 0.25인가? (Sigmoid 미분 그래프)



다층 구조에서의 전체 역전파

12

→ 순전파 (Forward)

입력 신호가 가중치와 곱해져 다음 층으로 전달됩니다.

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}$$

$$a^{(\ell)} = \phi(z^{(\ell)})$$

← 역전파 (Backward)

오차 신호(δ)가 거꾸로 흐르며 각 층의 책임을 분배합니다.

$$\delta^{(\ell)} = (W^{(\ell+1)})^T \delta^{(\ell+1)} \odot \phi'$$

GRADIENT CALCULATION

$$\partial L / \partial W^{(\ell)} = \delta^{(\ell)} (a^{(\ell-1)})^T$$

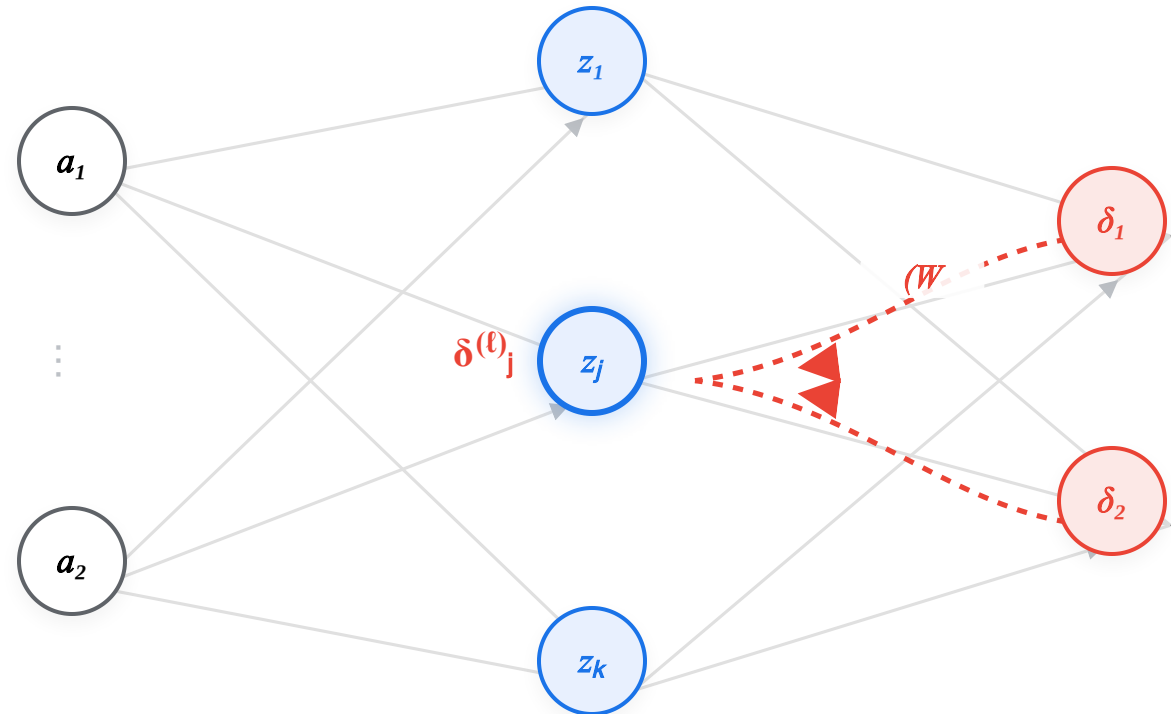
"현재 층의 오차(δ) × 이전 층의 입력(a)"

 $(\ell+1)^T$

Layer $\ell-1$
(입력)

Layer ℓ
(은닉)

Layer $\ell+1$
(출력/상위)



오차 신호 역류

상위 층의 모든 오차가
가중치와 곱해져
모입니다.

— 순전파 (정보) - - - 역전파 (오차 δ)

가중치 업데이트 공식의 시각적 의미

$$W_{\text{new}} = W_{\text{old}} - \eta \cdot \nabla L$$

↓ 방향: 마이너스 그라디언트 ($-\nabla L$)

기울기(Gradient, ∇L)는 손실함수의 값이 **가장 가파르게 증가하는 방향**을 가리킵니다.

우리는 손실을 줄여야 하므로, 그 반대 방향(-)으로 이동합니다.

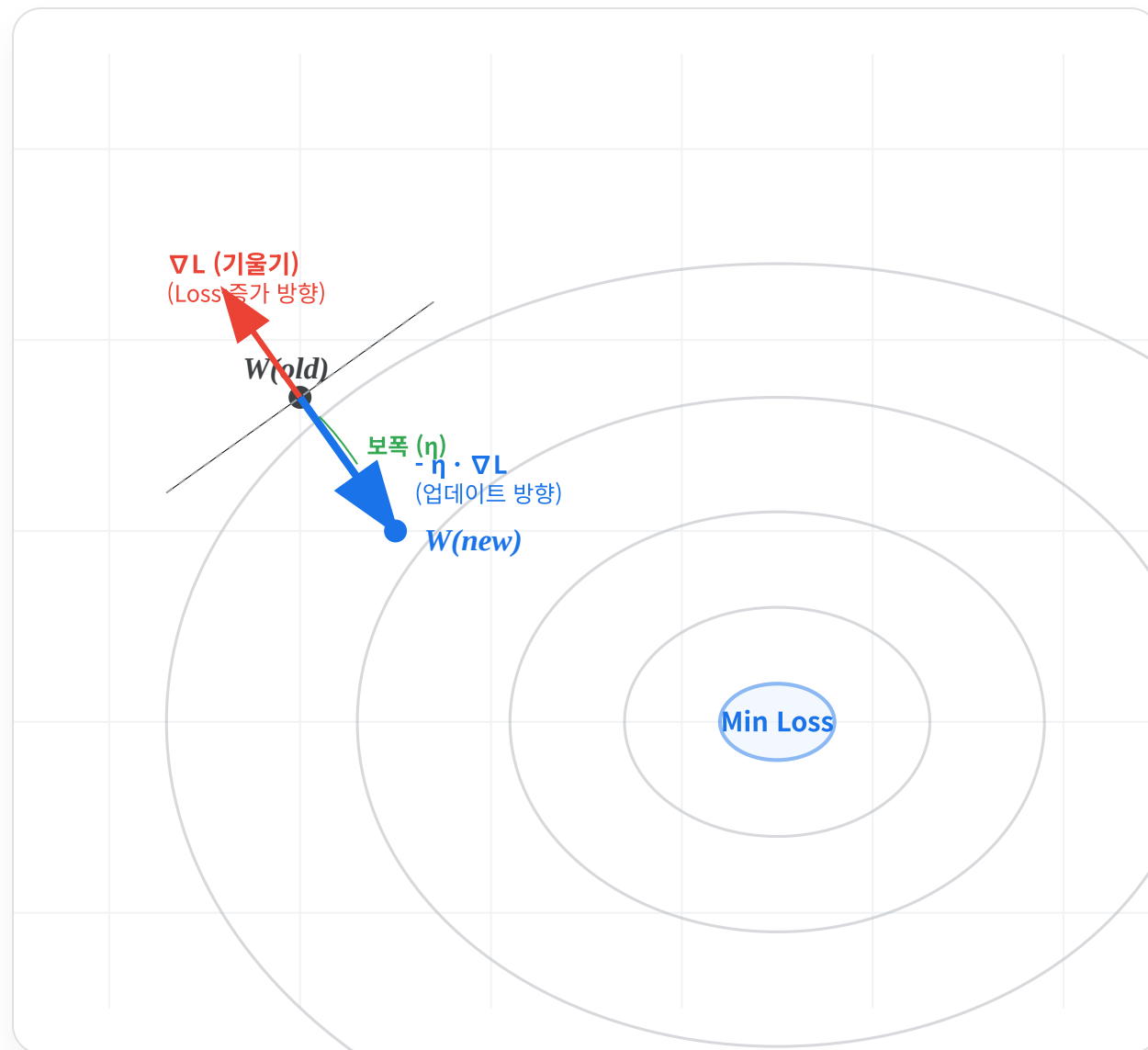
🔧 크기: 학습률 (Learning Rate, η)

한 번에 얼마나 이동할지 결정하는 보폭입니다.

너무 크면 최소점을 지나쳐 발산하고, 너무 작으면 학습 속도가 느려집니다.

🔄 동시 업데이트

이 공식은 네트워크 내의 모든 가중치(W)와 편향(b)에 대해 동시에 적용됩니다.



역전파의 효율성 (Efficiency)

수치 미분 (Numerical)

비효율적

$$\frac{\partial L}{\partial w_i} \approx \frac{L(w_i + \epsilon) - L(w_i)}{\epsilon}$$

- ❌ 파라미터가 100만 개라면, **100만 번의 순전파**를 다시 계산해야 합니다.
- 🕒 연산량: $O(P \times N)$
(P: 파라미터 수, N: 노드 수)

역전파 (Backprop)

효율적

Find all ∇L in one pass

- ✅ 중복 계산을 피하고, 단 **한 번의 역방향 스캔**으로 모든 기울기를 구합니다.
- ⚡ 연산량: $O(N)$
(순전파와 거의 동일한 비용)

연산 비용 비교 (Computational Cost)



수치 미분

파라미터 수(P)에 비례

작은

역전파

상수 시간 (순전파 × 2)

핵심 원리: 동적 계획법 (Dynamic Programming)



수치 미분

매번 처음부터 다시 계산
(중복 발생)



역전파

계산된 미분값(Gradient)을
저장하고 재사용

"경로상의 중간 결과를 **캐싱(Caching)**하여 상위 노드로 전달합니다."

단원 요약: 역전파 마스터하기

15



역전파의 본질

역전파는 '출력의 오차 책임'을 **연쇄 법칙(Chain Rule)**을 이용해 입력층까지 배분하는 효율적인 알고리즘입니다.



신호의 역류

 $\text{Gradient} \times \text{Local}$

계산 그래프 상에서 오차 신호(δ)는 뒤에서 앞으로 흐르며, 각 노드의 **국소적 미분**과 곱해집니다.



노드별 전파 규칙

덧셈(+)은 신호를 그대로 복제하여 전달하고, **곱셈($\times$)**은 서로의 입력값을 곱하여(스케일링하여) 교차 전달합니다.



활성화 함수의 역할

 $\sigma'(z)$

활성화 함수의 미분값은 역전파되는 기울기의 세기를 조절하는 **밸브** 역할을 합니다. (포화 시 기울기 소실 주의)



압도적인 효율성

수치 미분과 달리, 역전파는 단 한 번의 역방향 패스로 모든 파라미터의 기울기를 동시에 계산합니다.



학습의 완성

 $W \leftarrow W - \eta \nabla L$

구해진 기울기(∇L)의 반대 방향으로 가중치를 조금씩 이동시켜 손실을 줄입니다.

이제 딥러닝 학습의 엔진인 '역전파'를 완벽하게 이해했습니다. 다음 단계는 실전 구현입니다.